

Topic: Dotenv and JWT

Securing Configuration

AWS Secret Key

AWS Access Key

Azure identity key

Environment Variables

Small piece of data stored outside the source code.

Can be used to change how application works

.env file is used to store the environment variables

Another way to pass the environment variables and that is using the CLI.

dotenv automatically loads variable from .env file and ultimately into process.env

Authentication

- verifying the identity
 - prove who you say you are
 - using passwords, OTP, biometrics, OAuth

Authorization

- what are you allowed to do
- it is about permissions and access levels
 - what resources or actions can be accessed
 - based on roles, policies, scopes and rules
 - it happens after the authentication

JWT Tokens

JWT - JSON Web Token

compact, URL safe token for securely transmit the information between parties

Login flow

1. Token contains all the information needed for verification and server need not to save the session information.
2. Sent into the HTTP headers - Authorization Header
3. It is self contained - it has user data and a signature.

Token

eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.

eyJzdWl0Ij0IeXmjMONTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWRTaw4iOnRydWUsImhhdCI6MTUxNjIzOTAyNA
ANcf_8p1AE4ZQs7QuqG4yufTEgYrKSjKwkhBk5cIn1_2QVr2jEjmM-1tu7EqnyOf_fAsvdFXva8Sv05iTGzETg

3 parts of the JWT Token

1. Header - contains the metadata about the token
 - like algorithm used and the type of token
2. Payload - contains the claims (user info or any other data which is sent to the other party)
 - { id: 1, name: 'Alice', role: 'Admin' }
3. Signature - created by encoding the header and payload with a secret key
 - ensures the token is not tempered with.

`signature = hash_value(utf8(header) + utf8(payload), secret_value)`

How JWT works?

Example - Authentication Flow (Login)

1. User send the credentials
2. Server verify the credentials
3. Server generates JWT with payload and secret key
4. Server send the JWT back to the client
5. Client store the token in cookie, localStorage or sessionStorage
6. For every request client sends the JWT token along in the Authorization header
7. Server verifies the token using the secret key
8. If valid - access is granted else request is rejected